

CodeLens AI — Debugging & Trace Report

Generated on 7/29/2026 | Programming Language: **Java**

1. EXECUTION FLOW OVERVIEW

The program starts by declaring a method to calculate the average of an array of integers. It checks if the array is empty and returns 0.0 if so. Otherwise, it initializes a sum variable to 0.0 and iterates over the array, adding each element to the sum. After the loop, it calculates the average by dividing the sum by the array length and returns this value. In the main method, an example array is created, and the calculateAverage method is called with this array, printing the result.

Arrays **Loops** **Method Declaration** **Division** **Average Calculation**

2. ORIGINAL SOURCE CODE

```
public class Main {  
    public static double calculateAverage(int[] arr) {  
        if (arr.length <= 0) return 0.0;  
        double sum = 0.0;  
        for (int i = 0; i < arr.length; i++) {  
            sum += arr[i];  
        }  
        return sum / arr.length;  
    }  
  
    public static void main(String[] args) {  
        int[] numbers = {10, 20, 30, 40};  
        System.out.println("Average: " + calculateAverage(numbers));  
    }  
}
```

4. STEP-BY-STEP EXECUTION TRACES

Step #1: Method Declaration

Executing Line: public static double calculateAverage(int[] arr) {

Explanation: Declares a static method calculateAverage that takes an array of integers and returns a double value.

Under the Hood: The method declaration is stored in memory, and the Java Virtual Machine (JVM) adds it to the method call stack when invoked.

Analogy: Think of this method as a recipe book where you can look up how to make a specific dish, in this case, calculating an average.

Step #2: Array Length Check

Executing Line: `if (arr.length <= 0) return 0.0;`

Explanation: Checks if the input array is empty or has zero elements, returning 0.0 in such cases to avoid division by zero.

Under the Hood: The JVM checks the array length by accessing the length field of the array object in memory, and if true, it returns immediately, unwinding the call stack.

Analogy: It's like checking if you have any ingredients before trying to cook; if not, you can't proceed.

Step #3: Sum Initialization

Executing Line: `double sum = 0.0;`

Explanation: Initializes a variable sum to 0.0, which will be used to accumulate the sum of the array elements.

Under the Hood: The JVM allocates memory for the sum variable on the stack and initializes it to 0.0, preparing it for the upcoming loop.

Analogy: You start with an empty bucket to collect water; here, the bucket is the sum variable.

Step #4: Loop Initialization

Executing Line: `for (int i = 0; i < arr.length; i++) {`

Explanation: Starts a for loop that will iterate over each element of the input array.

Under the Hood: The JVM initializes the loop counter i to 0, checks the condition i < arr.length, and if true, proceeds to execute the loop body, allocating space on the stack for the loop variables.

Analogy: Imagine going through a list of items one by one, checking each item off as you go.

Step #5: Sum Accumulation

Executing Line: `sum += arr[i];`

Explanation: Adds the current array element to the sum variable in each iteration of the loop.

Under the Hood: The JVM loads the current array element `arr[i]` from memory, adds it to the sum variable, and stores the result back in the sum variable, updating the value on the stack.

Analogy: As you go through the list, you add each item's value to your bucket (sum), gradually filling it up.

Step #6: Average Calculation

Executing Line: `return sum / arr.length;`

Explanation: Calculates the average by dividing the sum of the array elements by the number of elements.

Under the Hood: The JVM performs the division operation, storing the result in a temporary location on the stack, and then returns this value, unwinding the call stack and freeing up memory.

Analogy: Finally, you divide the total value in your bucket by the number of items to find the average value per item.